Practical Workflow for Testing Visuomotor Robot Learning Systems

From MuJoCo Prototyping to Physical Deployment

Technical handover for the LIRMM DEXTER team

Prepared from:	the internship implementation by Luca Vogelgesang
Scientific supervision:	Chao Liu
Team:	LIRMM DEXTER
Version:	1.2
Date:	31 August 2026

Contents

Chapter 1

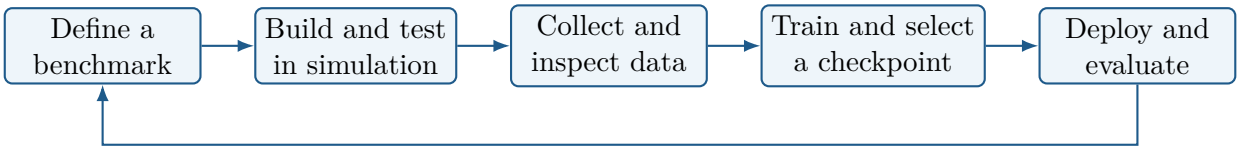
From an Idea to a Validated Simulation

The internship implementation appears only as a running example. The workflow itself applies to behavior cloning, action-chunking models, diffusion-based policies, or another vision-conditioned controller.

Use `README_ENV.md`, `README_SERVER.md`, and `README_CODE.md` for installation and copyable project commands; this guide focuses on the decisions and checks that remain useful across systems.

1.1 The experimental cycle

Most robot-learning projects can be organized as the following loop:



Each stage should produce evidence before the next one starts. A scene that loads is not yet a valid benchmark; a dataset that has the expected shape is not necessarily useful; a low validation loss is not proof of closed-loop task success; and a robot that moves is not yet a safe deployment.

General rule

Change one source of difficulty at a time. Begin with a task that is easy to interpret, then add position, orientation, lighting, object, or geometry variation progressively. When several variables change together, a failure is hard to diagnose and the next dataset is difficult to design.

1.2 Define success before writing code

A benchmark description should answer five questions:

1. What is the initial state distribution?
2. What observations are available to the policy?
3. What actions can the policy command?
4. What event counts as success or failure?
5. Which conditions remain fixed during one evaluation?

Avoid descriptions such as “the robot should usually grasp the object.” A useful definition is observable: for example, “the object starts inside a marked region and the trial succeeds when it is released inside the target box without triggering a safety stop.” Also define a maximum duration and the handling of contacts, re-grasps, human intervention, and sensor failure.

Internship example

The project moved from simulation to a simple physical pick-and-drop task, then to constrained extraction with progressively larger pose variation. Each stage added one observable difficulty while retaining the same experimental cycle.

1.3 Separate interfaces from implementations

A learned policy should be treated as one component with a clear contract:

$$\text{recent observations} \longrightarrow \text{future action sequence}. \tag{1.1}$$

The simulator and physical robot do not need the same internal code, but they must provide compatible concepts: camera images, robot state, action decoding, and timing. Keeping these interfaces explicit makes it possible to replace the model architecture or manipulator without rewriting the complete workflow.

Simulation can validate the method and interfaces without implying direct transfer of one trained model to the physical robot. Decide explicitly whether simulation data, model weights, or only software interfaces will be reused.

1.4 What should remain reproducible

For every dataset, training run, and evaluation, record at least:

- exact code version (the result of `gitrev-parseHEAD`) and configuration values used for the run;
- task definition and object-pose range;
- camera placement, crop, resolution, and exposure mode;
- robot start state and control mode;
- observation and action definitions;
- number of accepted demonstrations;
- selected checkpoint and deployment parameters;
- individual evaluation trials, not only the final percentage.

The exact command is part of the experiment. A checkpoint filename alone is insufficient because horizons, camera preprocessing, action convention, and safety limits may be supplied at runtime.

Before continuing

Write a one-page benchmark protocol and draw the observation/action interface. If success, variation, and action semantics are still ambiguous, do not begin a large data collection.

1.5 Build a Useful MuJoCo Benchmark

Simulation is most useful as an engineering instrument: it allows the task, controller, observations, recording format, and policy runner to be tested without risking hardware. It should be sufficiently representative to exercise the interfaces, but it does not need to reproduce every physical detail before the first experiment.

1.5.1 Start from the experimental question

Before editing XML, decide whether the simulator must validate reachability, camera information, teleoperation, data recording, or a complete policy rollout. Add contact detail, textures, and dynamics only when they represent a task requirement or explain a measured mismatch.

1.5.2 The MuJoCo concepts that matter in practice

MuJoCo describes the fixed system in MJCF/XML and stores the evolving state in `MjData` [Todorov et al., 2012]. Keep the following distinction in mind:
Model: bodies, joints, masses, geometry, cameras, actuators, and limits.
Data: joint state, control input, object pose, contact state, and time.
Forward computation: recomputes kinematics after directly changing state.
Simulation step: advances dynamics by one time step.

To randomize an object's pose at reset, modify the data and call forward kinematics. To change its mass or collision shape, modify the model description and reload it.

A robust scene-building order

Add the robot and a flat workspace first. Then add one primitive object, one camera, and one success region. Validate reachability and contacts before adding meshes, clutter, or multiple objects. Primitive collision geometry is usually more stable than a detailed visual mesh.

1.5.3 Example scene from the internship

The project scene is `dp_mujoco/models/universal_robots_ur10e/scene_microwave_camera.xml`. It separates scene geometry, robot description, environment access, and reset randomization. Keep the same separation in a new project, and ensure collection and execution use the same reset distribution.

Open any new scene before involving teleoperation:

```
conda activate microwave_dp
python -m mujoco.viewer \
  --mjcf=dp_mujoco/models/universal_robots_ur10e/scene_microwave_camera.xml
```

Verify limits, initial overlaps, collision geometry, camera placement, object settling, and repeated resets. Rendering without an exception is only the first check.

1.5.4 Design observations before collecting data

Camera choice should follow the task. A global view gives workspace context; a wrist view gives detail near contact but can be occluded by the gripper. Render exactly the images that the model will receive, including their crop and final resolution.

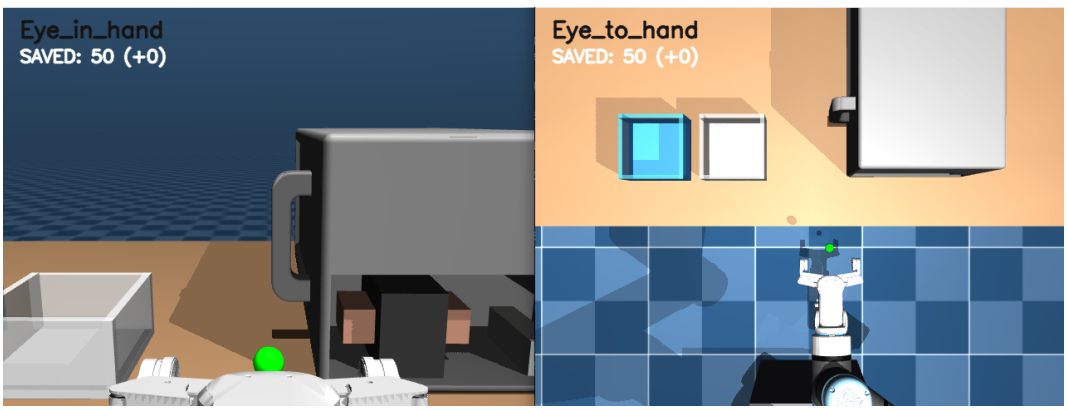


Figure 1.1: Simulation example: wrist view on the left and global view on the right.

Ask three questions for every camera:
1. Is the relevant object visible before the action decision is required?
2. Can the robot or gripper hide it during a critical phase?

3. Can approximately the same viewpoint be reproduced on the real setup?

Changing a camera, crop, or resolution after data collection changes the policy input distribution. Treat it as an interface change, not a cosmetic edit.

1.5.5 Add variation progressively

Randomization should represent the uncertainty that the policy is expected to handle. Define position and orientation ranges explicitly and sample only collision-free states. Test many resets before recording demonstrations.

Useful stages are:

1. fixed object and target positions;
2. one-dimensional position variation;
3. two-dimensional position variation;
4. limited orientation variation;
5. appearance, lighting, geometry, or object variation.

If the policy fails after a new stage, this ordering identifies which added uncertainty requires more data or a different observation.

1.5.6 Validate the full simulation loop

Touch pre-flight before the complete loop

`launch_all.sh` requires a working 3D Systems Touch input path. Before starting it, connect the device to the computer, power it when required, and confirm that the vendor Touch driver and OpenHaptics are installed. The launcher sources the ROS 2 workspace and builds it when generated files are missing; the first build still requires a valid `OPENHAPTICS_ROOT`.
For a first installation, follow appendix A and verify `/touch/pose` and `/touch/buttons` with the standalone ROS 2 driver before launching the complete teleoperation loop. Stop that standalone test before running `launch_all.sh`, which starts its own Touch node. For later sessions, plug in the device before launch and perform the daily terminal setup described in `README_ENV.md`.

In this project, simulation teleoperation is launched with:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
./launch_all.sh
```

The launcher starts the MuJoCo program and the ROS 2 Touch node. Before a large collection, record only three episodes and verify:

- the robot follows all Cartesian directions correctly;
- reset variation stays inside the protocol;
- both policy images update and have the expected resolution;
- actions and measured robot state evolve continuously;
- episodes can be saved, cancelled, and replayed.

Then train a small or short test run and execute one checkpoint in simulation. This smoke test finds interface errors more cheaply than a full training run.

1.5.7 Common mistakes and useful improvements

Symptom	First investigation
Unstable object contact	Check units, initial overlap, collision primitives, mass, and solver settings before changing control gains.
Black or irrelevant camera	Check camera name, transform, lighting, render update, crop, and final resized image.
Jump at reset	Clear velocities, apply the complete home/reset function, and recompute forward kinematics.
Collection and rollout differ	Compare randomization, camera order, quaternion convention, image shape, and action scaling.
Simulation is too easy	Increase one measured uncertainty at a time rather than adding arbitrary noise everywhere.
Simulation is too slow	Profile rendering, physics, and policy inference separately; they require different fixes.

Ready for demonstration collection

The task can be completed repeatedly by teleoperation, every reset is valid, the policy observations contain the required information, and a saved episode can be replayed without shape or convention ambiguity.

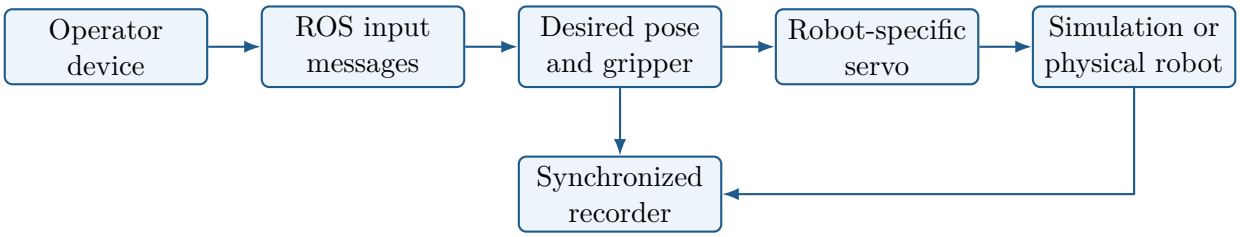
Chapter 2

Teleoperation and Data Recording

Teleoperation is not only a convenient way to move the robot. It defines the behavior that the policy will imitate. Controller lag, inconsistent gripper commands, occlusion, and unnecessary corrective motions all become part of the training target.

2.1 A reusable teleoperation architecture

A practical teleoperation system separates device input, target generation, low-level control, and recording:



This separation makes the input device and learned-policy interface reusable. Only the servo and state reader must change for another robot.

Implementation used during the internship

The 3D Systems Touch publishes pose and buttons through a ROS 2 C++ node. A Python listener converts incremental stylus motion into a Cartesian target. Pinocchio computes forward kinematics and the Jacobian [Carpentier et al., 2019]. The same target logic feeds either MuJoCo or a UR10 `speedj` backend.

2.2 Use relative motion, not device coordinates

The physical coordinates of a desktop device generally have no useful absolute relationship with the robot base. Use differential motion:

$$\Delta p_{robot} = s_p M_p \left(p_t^{device} - p_{t-1}^{device} \right), \tag{2.1}$$

where M_p maps axes and s_p sets workspace scale. The first valid device pose becomes a reference and produces no robot movement. This prevents the initial jump that would occur if the device pose were interpreted as an absolute robot target.

Test one translation and one rotation axis at a time. If an axis is wrong, correct the mapping rather than teaching the operator to compensate mentally.

2.3 From target error to robot motion

For Cartesian velocity control, a common structure is:

$$v_{des} = \begin{bmatrix} K_p e_p & K_r e_r \end{bmatrix}^T, \qquad \dot{q} = J^{\#}(q) v_{des}, \tag{2.2}$$

where e_p and e_r are pose errors and $J^{\#}$ is a damped pseudoinverse. Joint velocity and acceleration limits must be applied after this conversion.

Target smoothing can reduce abrupt hand motion:

$$x_f(t) = (1 - \alpha) x_f(t - 1) + \alpha x_{raw}(t). \tag{2.3}$$

A large α is responsive but transmits more noise; a small α is smooth but introduces lag. Tune scale, target-speed limit, filter alpha, servo gain, and joint limit one at a time. Several different parameter combinations can produce similar slow motion while having very different behavior near contact.

2.4 Understand the timing chain

The device, controller, cameras, and recorder do not need the same frequency. For example, the internship setup approximately used:

Table 2.1: Typical rates in the real teleoperation pipeline.

Process	Reference rate
Touch ROS 2 publication	approximately 100 Hz
Robot control loop	50 Hz
RealSense capture	30 Hz
Demonstration recording	10 Hz

The controller should consume the newest available target, while the recorder periodically samples a coherent observation-action pair. Queueing every old device message increases lag without adding useful information.

Measure latency instead of guessing

Timestamp device input, camera capture, target generation, command transmission, and recorder write. Log dropped or stale frames. A slow response may come from the ROS path, camera blocking, controller filtering, disk compression, or the robot safety slider; increasing K_p does not fix all of these causes.

Useful improvements for a future implementation are:

- use depth-one/latest-sample ROS QoS for interactive targets;
- capture cameras continuously rather than starting a blocking read in the servo loop;
- move image compression and Zarr writes to a bounded background queue;
- record timestamps and reject a sample when one camera is stale;
- display control error, frame age, and saved-step count to the operator;
- stop or invalidate the episode after a frozen camera rather than saving apparently normal repeated images.

2.5 Design the gripper command as part of the task

Continuous gripper measurements are not always good action labels. A physical gripper moves slowly and may report different intermediate widths for the same operator intention. If the task naturally uses semantic states, record the commanded state as the action and the measured width as an observation.

In the internship dataset, the gripper action used three semantic states (open, narrow, and grasp). The narrowed state allowed entry into the cavity. The commanded state was stored in `action`, while measured opening was stored in `robot0_gripper_qpos`; this preserved operator intention and still exposed actuator delay.

Use continuous commands when the task genuinely requires variable grasp width. Do not quantize only to hide inconsistent data; choose an action representation that matches the future task.

2.6 Collect demonstrations that teach observable corrections

A good demonstration is not simply a successful one. It should make the reason for each correction visible to the policy. Practical recommendations are:

- practice the controller before recording;
- begin and end episodes close to the useful task interval;
- use a consistent phase order and start configuration;
- avoid moving from information that is visible only to the human;
- delay fine alignment until the relevant camera sees the object;
- avoid passing the gripper in front of the object unnecessarily;
- cancel failed, interrupted, or camera-corrupted demonstrations;
- include deliberate coverage of the expected pose range.

In the constrained-extraction experiments, early demonstrations sometimes began orientation corrections before the wrist camera clearly saw the component. Later demonstrations approached first, waited for a useful wrist view, and then aligned. This made the observation-action relationship easier to learn.

2.7 Launch through maintained wrappers

The repository uses `launch_all.sh` for simulation and `ur10_real_robot/run_teleop.sh` for physical collection. The complete commands and controls are maintained in `README_CODE.md`. On another platform, replace the robot adapter and limits while preserving the staged pre-flight, recording, and dataset checks.

Physical teleoperation

An operator must be trained on the platform, the speed slider must begin low, the workspace must be clear, and an emergency stop must remain reachable. Test each mapped axis with millimetric motions before combining directions or recording data.

Before a long collection

Record three episodes, replay every frame, plot the trajectory, and verify the gripper labels. Continue only if the motion is responsive enough for small corrections and the saved data represents what the operator intended.

Chapter 3

From Data to a Trained Model

The dataset is the contract between teleoperation, training, and deployment. Most difficult bugs in this transition are not neural-network bugs; they are differences in key names, image order, normalization, coordinates, quaternion convention, timing, or action semantics.

3.1 Define one synchronized sample

A visuomotor sample normally contains:

$$(o_t, a_t) = (\text{images}_t, \text{robot state}_t, \text{demonstrated command}_t)\,. \tag{3.1}$$

An episode is an ordered sequence of these samples from one task attempt. Do not concatenate unrelated attempts without episode boundaries; training windows must not cross from the end of one trial into the start of another.

The recorder should define which timestamp each field represents. For a slow 10 Hz dataset, a camera frame that is several hundred milliseconds older than the action may already create a misleading label near contact.

3.2 Define and document the schema

The exact keys are project-specific. For example, the internship recorder stored the following Zarr arrays at 10 Hz:

Table 3.1: Real dataset interface used during the internship.

Key	Per-step shape	Meaning
`agentview_image`	$240 \times 320 \times 3$	Global RGB image.
`robot0_eye_in_hand_image`	$240 \times 320 \times 3$	Wrist RGB image.
`robot0_eef_pos`	3	Measured end-effector position.
`robot0_eef_quat`	4	Measured end-effector quaternion.
`robot0_gripper_qpos`	1	Measured gripper opening/state.
`action`	8	Target pose and gripper command.

The real action was

$$a_t = [x, y, z, q_w, q_x, q_y, q_z, g]. \tag{3.2}$$

Dimension alone is insufficient compatibility evidence: coordinate frames, units, quaternion order, and action meaning must also match.

3.3 Choose state and action according to causality

The observation should describe what was measured before the command. The action should describe what the demonstrator intended the robot to do next. This distinction is important for slow actuators:

- measured gripper width belongs in the observation;
- commanded open/narrow/grasp state belongs in the action;
- delayed measured motion should not replace the intended command label.

Apply the same reasoning to target pose versus measured pose. If the action is a Cartesian target, record that target consistently during collection and decode it the same way during execution.

3.4 Temporal windows and horizons

Sequence policies use several time scales:
Observation horizon T_o: recent samples supplied as context.
Prediction horizon T_p: complete future sequence predicted by the model.
Action horizon T_a: prefix executed before replanning.

For a sequence policy, predicting farther than execution can encourage a coherent future while replanning limits the use of increasingly old predictions. Record all three horizons with the model configuration.

These horizons are part of the model interface. Changing them may require a new configuration or checkpoint rather than only a command-line flag.

3.5 Inspect structure, content, and coverage

Dataset validation should occur in three passes.

3.5.1 Structural checks

Confirm episode count, array lengths, dtypes, action dimension, image shapes, and absence of incomplete chunks. In this repository:

```
python scripts/check_zarr.py data/datasets/<dataset>.zarr
```

3.5.2 Visual and trajectory replay

Replay every episode, not only random images:

```
python scripts/replay_zarr.py data/datasets/<dataset>.zarr --scale 2
python scripts/traj_zarr.py data/datasets/<dataset>.zarr
```

Look for frozen cameras, repeated frames, occlusion at the decision point, sudden action jumps, long idle segments, wrong gripper transitions, and recovery motions that are not intended behavior.

3.5.3 Coverage checks

A collection can contain only good demonstrations and still be a poor dataset. Plot or tabulate initial position, orientation, task zone, object identity, and lighting condition. The distribution should cover the evaluation protocol rather than accidentally overrepresenting the easiest location.

Use data inspection to redesign collection

When a policy repeatedly misses one region, first check whether that region is visible and sufficiently represented. Additional targeted demonstrations may be more useful than changing architecture or training for more epochs.

3.6 Compare deployment observations with training data

Before executing a trained model after the setup has moved, compare live camera images with recorded images. This repository provides:

```
PYTHONPATH=. python scripts/compare_live_camera_to_zarr.py \
  data/datasets/<dataset>.zarr \
  --wrist-camera-config <matching_camera_config.json>
```

The views do not need to be pixel-identical, but camera geometry, crop, workspace position, and illumination should be close. A moved table or camera can create a larger distribution shift than a change in neural-network loss.

3.7 Training, validation, and evaluation data

Split by complete episodes, never by individual frames. Adjacent frames from one trajectory are highly correlated; placing some in training and some in validation gives an overly optimistic metric.

Validation loss is useful for detecting optimization problems and overfitting, but it does not directly measure closed-loop recovery. Select final candidates using held-out metrics and controlled rollouts, then freeze one model before the reported evaluation.

3.8 Acceptance checklist before training

Interface freeze

Confirm shapes and episode boundaries, replay every accepted episode, document camera and coordinate conventions, verify action semantics and coverage, then point the task configuration to this exact dataset. Keep one interface table for both training and deployment.

3.9 Train and Select a Model on Shared Compute

Training should begin only after the dataset interface has been frozen and a small end-to-end smoke test has succeeded. The goal of the server workflow is not merely to obtain a checkpoint; it is to preserve enough information to reproduce how that checkpoint was produced.

3.9.1 Freeze one training configuration

Record the following before launching a long run:

- dataset path and episode count;
- observation keys, shapes, and preprocessing;
- action representation and horizons;
- model architecture and parameter count;
- optimizer, learning rate, scheduler, batch size, and seed;
- train/validation split by episode;
- number of epochs and checkpoint interval;

- software revision and training device.

For a new model, first train for only a few batches and load the resulting checkpoint in the policy runner. This verifies configuration composition, tensor shapes, normalization, and serialization before consuming hours of GPU time.

3.9.2 Keep execution contexts explicit

Distinguish the local experiment PC, server host, container, and Python environment. Many path and import errors come from running a correct command in the wrong context; for example, `/workspace` exists inside the project container, not on the local robot computer.

3.9.3 A general server workflow

1. Validate the dataset and record the source revision.
2. Transfer only the required source, configuration, and data.
3. Use a persistent session and a documented container/environment.
4. Print resolved paths and run a short smoke test.
5. Monitor the run and copy model candidates with their configurations back to the deployment computer.

Use a persistent container or a documented image. Installing arbitrary packages interactively until an import works makes the next run difficult to reproduce.

3.9.4 Example: LIRMM training setup

Internship example

Training ran on `mic-gpu1` inside a persistent Docker container using one NVIDIA A100 GPU. Data and configurations were transferred with `rsync`; `tmux` protected the run from SSH disconnection. An absolute dataset path was supplied explicitly, and the trained model was loaded on the local CPU before robot testing. Exact server and training commands are in `README_SERVER.md`.

3.9.5 What to monitor

Monitor several signals rather than one number:
Training loss: confirms that the optimizer can fit the demonstrations.
Validation loss: detects generalization degradation on held-out episodes.
Action error or samples: helps reveal one problematic action component.
GPU use and step time: reveals data-loader, memory, or device problems.
Closed-loop rollouts: measure the behavior that ultimately matters.

The first Zarr load may be slow because chunks are decompressed and cached. A pause after model construction is not proof that training has frozen; inspect CPU, disk, GPU, and process activity before interrupting it.

3.9.6 Select model candidates

Save model candidates often enough to observe overfitting. Validation loss may reach its minimum early, while closed-loop performance does not always rank candidates in exactly the same order.

A sensible selection procedure is:

1. reject runs with unstable or inconsistent training;
2. choose a small set around the best validation region;
3. benchmark inference time and load each checkpoint locally;
4. test candidates in simulation or a no-motion real stage;

5. choose one checkpoint before the reported evaluation trials.

Do not change the selected model during the final evaluation. Development trials and evaluation trials have different roles.

3.9.7 Typical failures

Symptom	First checks
Dataset not found	Print the path inside the container; verify mount, transfer location, and Hydra-resolved value.
CUDA out of memory	Check other processes, selected GPU, batch size, workers, image resolution, and model size.
Training appears frozen	Inspect first-load caching, disk activity, workers, shared memory, and GPU utilization.
Loss becomes NaN	Check normalization, invalid actions, learning rate, mixed precision, and exploding gradients.
Validation rises early	Check split size, dataset diversity, augmentation, checkpoint frequency, and overfitting.
Checkpoint fails locally	Check source revision, policy class, preprocessing, dependency compatibility, and EMA loading.

Training output

For every candidate checkpoint, keep the resolved configuration, dataset name, Git revision, epoch, validation metrics, parameter count, and a measured inference latency on the deployment computer.

Chapter 4

Deploying a Policy on a Robot

Deployment connects a statistical model to a safety-critical dynamical system. Treat it as an interface and validation problem, not as one command that changes `device=cpu` to `device=cuda`.

4.1 Implement a robot adapter

For a new robot, isolate four responsibilities:

1. build observations in the exact training format;
2. load normalization and run the policy;
3. decode predicted actions into Cartesian or joint targets;
4. enforce low-level control limits and stop conditions.

The policy should not directly open sockets, read cameras, or implement robot kinematics. A robot-specific adapter makes it possible to reuse the same policy runner and replace only state acquisition and command execution.

UR10 implementation

`ur10_real_robot/policy_exec/` contains the observation builder and real runner. The backend reads UR10 state, two RealSense cameras, and gripper width; a Pinocchio Cartesian servo converts pose targets to bounded joint velocities; the gripper uses a Modbus adapter.

4.2 Deploy in stages

Never begin with unrestricted physical motion. Use at least these stages:

1. **Offline/fake:** load the checkpoint, create correctly shaped dummy observations, and inspect predicted actions.
2. **Real observations, no motion:** connect cameras and state readers, run inference, and print targets while commands remain disabled.
3. **Low-speed physical motion:** use conservative limits, one object pose, a clear workspace, and an operator ready to stop.
4. **Frozen evaluation:** fix checkpoint and parameters, then execute the written protocol.

At each stage, compare current observations with training data and verify units, frames, quaternion order, TCP offset, action horizon, and gripper convention.

4.3 Synchronous and asynchronous inference

If inference is slower than action execution, synchronous replanning causes the robot to pause while each new sequence is predicted. Asynchronous execution can hide this delay:

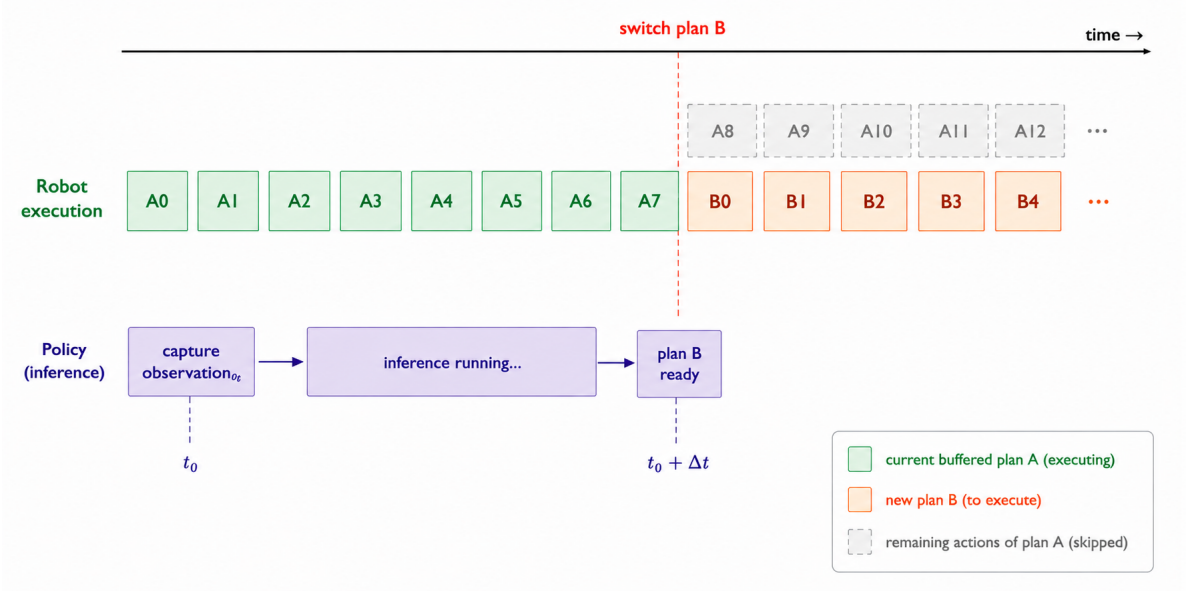


Figure 4.1: Asynchronous receding-horizon execution. The current buffered plan continues while the next plan is computed, then execution switches when the new plan is ready.

This improves continuity but does not remove latency. The new plan is based on an observation captured before inference began, so it may already be stale when execution starts. Faster robot motion, longer inference, and contact-rich tasks increase this mismatch.

Measure:

- policy inference latency and its variance;
- age of the observation used for each plan;
- remaining actions when a new plan becomes ready;
- target discontinuity at plan switches;
- camera and control-loop delays separately.

Possible improvements include a deployment accelerator, a smaller model, a faster valid decoder or sampler, temporal ensembling, latency-aware action selection, and blending compatible plan transitions. Revalidate behavior after changing model-specific inference settings.

4.4 Parameters have different responsibilities

Do not tune every parameter as a generic “speed” control.

Table 4.1: Main deployment parameter groups.

Group	**What it controls**
Model decoding/sampling	Computation cost and output approximation.
Action/execution horizon	How long one prediction is consumed before replacement.
Action execution rate	Time spacing between buffered targets.
Cartesian gains	How aggressively tracking error is corrected.
Target smoothing	Discontinuity/noise versus response lag.
Joint and Cartesian limits	Maximum physical motion allowed by software.
Position/rotation stop thresholds	Maximum discrepancy accepted before stopping.
Teach-pendant slider	Hardware-level scaling of physical speed.

Record both requested software limits and the teach-pendant slider. Two runs with the same command but different slider values are not the same execution condition.

4.5 Safety stops should latch and explain themselves

A stop should cancel motion, prevent the same invalid target from being sent repeatedly, and print enough context to diagnose the cause. Useful checks include:

- target position and orientation error;
- joint velocity and workspace limits;
- stale camera or robot state;
- excessive control-loop time;
- loss of robot or gripper communication;
- invalid or non-finite policy output.

Do not increase a threshold only because it interrupts a trial. First determine whether the large error comes from a wrong frame, quaternion, delayed plan, unreachable target, or genuine tracking lag.

4.6 Camera and gripper behavior during deployment

Use the same camera order, crop, resolution, and exposure strategy as training. Warm up cameras before the first observation and reject stale frames. If the scene requires auto-exposure, give the camera time to adapt in both demonstrations and deployment.

Decode the gripper action using the same semantics used during recording. Log raw policy output, post-processing, command sent, target width, and measured width. This distinguishes policy hesitation from actuator delay or contact.

4.7 Use a staged execution wrapper

The repository wrapper `ur10_real_robot/run_diffusion_real.sh` defaults to motion disabled and requires an explicit motion-enable confirmation. Its validated project commands are in `README_CODE.md`. A new robot adapter should expose equivalent fake, observation-only, and physical-motion stages.

Learned policy execution

A successful checkpoint load does not imply a safe target. Keep independent software limits, robot safety configuration, low initial speed, an unobstructed workspace, and an operator able to stop motion immediately.

Ready for evaluation

The selected checkpoint and command are frozen; camera images match the dataset; timing has been measured; repeated low-speed trials show bounded tracking; and every stop condition has been tested or reviewed.

Chapter 5

Evaluation, Diagnosis, and Iteration

Evaluation should answer a defined question about one frozen system. It should not be mixed with checkpoint selection, gain tuning, or collection of new demonstrations.

5.1 Freeze the protocol

Before reported trials, fix:

- checkpoint and policy configuration;
- deployment command and safety limits;
- robot start state and gripper opening;
- camera placement, crop, and lighting;
- object/target distribution;
- trial success and failure rules;
- maximum time and permitted human intervention.

Development trials used to select these choices are excluded from the final results. If a parameter changes, start a new evaluation block.

5.2 Choose metrics that reveal task structure

Task success is necessary but often insufficient. Divide the task into phases, for example approach, alignment, grasp, extraction, transport, and release. Record where the first unrecovered failure occurs, whether a re-grasp was needed, whether contact occurred, and whether a safety stop triggered.

A 20-trial evaluation is useful for a preliminary engineering comparison, but it does not establish broad robustness or a precise success probability. Preserve individual trial records so later work can pool or stratify evidence.

Internship example

Evaluation difficulty was increased from a simple physical pick-and-drop task to constrained extraction with position and orientation variation. Each stage used a newly frozen dataset/-model pair, so the results measured feasibility at that stage rather than zero-shot transfer of one unchanged policy.

5.3 Diagnose the layer that failed

Classify evidence before changing the model:

Layer	Typical evidence and next check
Perception	Failure depends on lighting, occlusion, crop, or region; replay the exact policy images and compare coverage.
Demonstration	Unnecessary corrections or inconsistent grasp timing appear in training episodes; curate or recollect targeted data.
Model/training	Open-loop predictions are unstable across held-out episodes or seeds; inspect normalization, capacity, loss, and overfitting.
Timing	Behavior degrades with speed or asynchronous delay; measure observation age and plan-switch discontinuity.
Controller	Targets are reasonable but tracking overshoots or lags; inspect frames, gains, limits, and kinematics.
Gripper/contact	Arm motion is correct but measured width or grasp differs; inspect command semantics, actuator delay, force, and collision.
Hardware/sensor	Frames freeze or connections fail; repair the acquisition path before evaluating the policy.

Prefer the smallest informative experiment

If one zone fails, compare that zone with another while freezing checkpoint, lighting, and speed. If asynchronous execution is suspected, compare sync and async at the same pose. A controlled pair of trials can be more informative than immediately collecting another 50 demonstrations.

5.4 Improve data with human intervention

When the policy fails in a reproducible state, a human can demonstrate the missing correction and add it to a new dataset version. This is more targeted than recollecting the entire task, but new data must be balanced so that the policy does not forget common successful behavior. Human-in-the-loop and intervention-based approaches formalize this idea [Mandlekar et al., 2020, Ross et al., 2011].

Keep dataset versions immutable. Create a new version that records which episodes were added, why they were added, and which evaluation failure motivated them.

5.5 Starting a new model or robot

For a new policy architecture, preserve the dataset and evaluation protocol first. Verify that the model accepts the same interface and compare parameter count, training stability, inference latency, and task success. Architecture comparison is not meaningful if observations, action horizon, or evaluation conditions also change.

For a new robot, preserve the high-level benchmark but revalidate:

- kinematic reachability and tool frame;
- camera geometry and occlusion;
- action/controller interface and timing;
- gripper semantics and force capability;
- workspace and safety limits;
- whether old demonstrations remain physically meaningful.

The most reusable artifacts are the benchmark definition, data schema, replay tools, evaluation sheet, and staged deployment method. Robot-specific gains and network addresses are not reusable scientific conclusions.

5.6 Minimal handover record

At the end of one experiment, leave:

1. a short task protocol;
2. dataset path, episode count, and quality notes;
3. resolved training configuration and source revision;
4. selected checkpoint and measured inference latency;
5. exact deployment command;
6. trial-level evaluation table and representative videos;
7. known failure cases and the next controlled experiment.

This is enough for another student to understand what was tested, reproduce the main result, and continue the investigation without repeating the full development history.

Chapter A

Touch and OpenHaptics Installation

This appendix gives the complete device-specific installation used for the 3D Systems Touch. The validated stack is Ubuntu 22.04, ROS 2 Humble, OpenHaptics 3.4, and Touch Driver 2025.12.10. Install the general local environment from `README_ENV.md` first. Keep the Touch disconnected until the driver installation is complete.

A.1 Download and extract the vendor packages

Download both Linux archives from the official 3D Systems support page:

https://support.3dsystems.com/s/article/OpenHaptics-for-Linux-Developer-Edition-v34?language=en_US

- **OpenHaptics Installer — OpenHaptics for Linux v3.4**;
- **Haptic Device Driver Installer — Touch Device Driver v2025.12.10**.

Create a permanent working directory and extract both archives. The browser may append a suffix such as `(1)` to a repeated download; adapt the archive name when necessary.

```
mkdir -p "$HOME/touch_driver"

tar -xzf "$HOME/Downloads/openhaptics_3.4-0-developer-edition-amd64.tar.gz" \
  -C "$HOME/touch_driver"
tar -xzf "$HOME/Downloads/TouchDriver_2025_12_10+1.tgz" \
  -C "$HOME/touch_driver"

ls "$HOME/touch_driver/TouchDriver_2025_12_10"
ls "$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"
```

Do not place these vendor archives inside the Git repository.

A.2 Install dependencies and vendor software

Install the build, terminal, and OpenGL dependencies required by the SDK and its examples:

```
sudo apt update
sudo apt install -y \
  build-essential freeglut3-dev libglu1-mesa-dev \
  libncurses5-dev mesa-utils
```

Install the Touch driver first. This copies the device libraries and installs the USB permission rule:

```
cd "$HOME/touch_driver/TouchDriver_2025_12_10"
sudo bash ./install_haptic_driver
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Reconnect the Touch after this command. Reboot if requested. Then install the OpenHaptics SDK:

```
cd "$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"
sudo ./install
sudo ldconfig
```

The SDK installer places examples and documentation under `/opt/OpenHaptics/Developer/3.4-0/`. The project also keeps the extracted SDK root so that CMake can locate the exact headers and libraries.

A.3 Configure the SDK path

Define `OPENHAPTICS_ROOT` and verify the two files required by the project build:

```
export OPENHAPTICS_ROOT="$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"

test -f "$OPENHAPTICS_ROOT/usr/include/HD/hd.h" \
  && echo "OpenHaptics headers found"
test -f "$OPENHAPTICS_ROOT/usr/lib/libHD.so" \
  && echo "OpenHaptics HD library found"
```

Both messages must appear. Make the setting persistent for Bash terminals:

```
echo 'export OPENHAPTICS_ROOT="$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"' \
  >> "$HOME/.bashrc"
source "$HOME/.bashrc"
```

For Zsh, add the same export line to `~/.zshrc` instead.

Some vendor executables may report missing legacy terminal libraries such as `libtinfo.so.5` or `libncurses.so.5`. Install them only when that error occurs:

```
sudo apt install -y libtinfo5 libncurses5
```

A.4 Validate the device before ROS

Plug in the Touch. First compile and run the read-only vendor diagnostic:

```
cd "$OPENHAPTICS_ROOT/opt/OpenHaptics/Developer/3.4-0/examples/HD/console/QueryDevice"
make
./QueryDevice
```

The program must identify the device, and the displayed pose and button values must change when the stylus moves. If initialization fails, stop here and fix the driver, USB connection, permissions, or missing library before using ROS.

Run the calibration example when the device requests calibration:

```
cd "$OPENHAPTICS_ROOT/opt/OpenHaptics/Developer/3.4-0/examples/HD/console/Calibration"
make
./Calibration
```

Finally, an optional force-feedback test verifies bidirectional communication:

```
cd "$OPENHAPTICS_ROOT/opt/OpenHaptics/Developer/3.4-0/examples/HD/console/FrictionlessSphere"
make
./FrictionlessSphere
```

Move the stylus gently; a virtual sphere should be perceptible. Keep a light grip and stop immediately with `Ctrl+C` if the device behaves unexpectedly.

A.5 Build the project ROS 2 driver

Activate Conda, ROS 2, and the SDK path before building the workspace:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
source /opt/ros/humble/setup.bash
export OPENHAPTICS_ROOT="$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"

cd ros2_WS
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install \
```

```
  --cmake-args -DOPENHAPTICS_ROOT="$OPENHAPTICS_ROOT"
source install/setup.bash
```

If the workspace was copied from another computer, old absolute paths may remain in generated files. Remove only the generated directories and rebuild:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim/ros2_WS"
rm -rf build install log
colcon build --symlink-install \
  --cmake-args -DOPENHAPTICS_ROOT="$OPENHAPTICS_ROOT"
source install/setup.bash
```

A.6 Validate the ROS topics

Start the Touch node in terminal 1:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
source /opt/ros/humble/setup.bash
source ros2_WS/install/setup.bash
ros2 launch touch_ros2_driver touch_rviz.launch.py
```

Inspect its outputs in terminal 2:

```
source /opt/ros/humble/setup.bash
source "$HOME/Stage_Lirmm/Diffusion-model-isaacsim/ros2_WS/install/setup.bash"
ros2 topic hz /touch/pose
ros2 topic echo /touch/buttons
```

The pose must update continuously and both buttons must change the published value. Stop both terminals with `Ctrl+C`. Do not continue to MuJoCo or a physical robot if the topic freezes, the axes jump while the device is still, or the node reports an OpenHaptics scheduler error.

A.7 Daily use and common failures

In every new terminal used with the Touch, activate the three software layers:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
source /opt/ros/humble/setup.bash
source ros2_WS/install/setup.bash
```

Symptom	Check
Device not detected	Reconnect USB, reload the udev rules, reboot, and run `QueryDevice` before ROS.
Missing legacy terminal library	Install the two compatibility packages shown above.
CMake cannot find `HD/hd.h`	Correct `OPENHAPTICS_ROOT`, then delete only `ros2_WS/build`, `install`, and `log` before rebuilding.
Scheduler or device-in-use error	Close other OpenHaptics examples and ROS nodes; only one process should own the device.
ROS topics are absent	Source ROS 2 and `ros2_WS/install/setup.bash`, then inspect the first terminal for a driver error.

Only after the vendor diagnostics and ROS topics pass should the Touch be linked to simulation or robot teleoperation. Physical robot motion still requires the staged safety procedure in chapter 2.

Bibliography

Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiraux, Olivier Stasse, and Nicolas Mansard. The pinocchio c++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *IEEE/SICE International Symposium on System Integration*, pages 614–619, 2019.

Ajay Mandlekar, Danfei Xu, Roberto Martín-Martín, and Silvio Savarese. Learning to generalize across long-horizon tasks from human demonstrations. In *Robotics: Science and Systems*, 2020.

Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.